Raport Proiect - Aplicație de Gestionare Filme și Recenzii

Autor: Sipanu Eduard Nicusor **Data:** 12 Ianuarie 2026 **Limbaj:** Java (Spring Boot) **Baza de date:** SQL Server

---

Cuprins

1. Specificare Funcționalități
2. Descrierea Claselor, Metodelor și Atributelor
3. Interfața Grafică cu Utilizatorul
4. Testarea Aplicației
5. Îmbunătățiri Propuse

---

1. Specificare Funcționalități

1.1 Funcționalități de Bază

Autentificare și Autorizare Aplicația implementează un sistem complet de autentificare bazat pe sesiuni HTTP. Utilizatorii pot crea conturi noi prin formularul de înregistrare, care validează unicitatea username-ului și email-ului. Autentificarea se realizează prin verificarea credențialelor în baza de date, iar sesiunea este stocată pe server. Toate rutele principale sunt protejate și redirecționează utilizatorii neautentificați către pagina de login.

Gestionare Filme Sistemul permite vizualizarea filmelor sub formă de listă cu carduri individuale. Fiecare film are o pagină dedicată care afișează informații detaliate: titlu, an apariție, durată, descriere, genuri, actori, regizori și rating mediu.

Funcționalitatea de căutare permite filtrarea după patru criterii: - Titlu film - Gen - Actor (nume sau prenume) - Regizor (nume sau prenume)

Sortarea este disponibilă după rating, titlu, an apariție sau durată, în ordine crescătoare sau descrescătoare.

Gestionare Recenzii Aplicația oferă operațiuni CRUD complete pentru recenzii:

Create: Utilizatorii pot adăuga recenzii cu titlu opțional, text obligatoriu și rating între 1-10. Sistemul verifică dacă utilizatorul are deja o recenzie pentru filmul respectiv și previne duplicatele.

Read: Toate recenziile pentru un film sunt afișate pe pagina filmului, incluzând autorul, data postării, rating-ul și textul complet.

Update: Utilizatorii pot edita propriile recenzii prin intermediul unui formular inline care se activează la click pe butonul de editare. Formularul este pre-populat cu datele existente.

Delete: Ștergerea recenziilor este protejată printr-un dialog de confirmare JavaScript care previne ștergerea accidentală. Doar autorul poate șterge propria recenzie.

1.2 Funcționalități Suplimentare

Rating Incremental Avansat Sistemul implementează o metodă avansată de actualizare a rating-ului filmelor. În loc să recalculeze media tuturor recenziilor la fiecare modificare, rating-ul se actualizează incremental folosind formula:

```
rating_nou = rating_curent + (review_rating - rating_curent) / 100
```

Această abordare oferă următoarele avantaje: - Performanță superioară (O(1) în loc de O(n)) - Rating-ul evoluează gradual, fără salturi bruște - Consistență între operațiuni de adăugare și ștergere

La ștergerea unei recenzii, se aplică formula inversă:

```
rating_nou = rating_curent - (review_rating - rating_curent) / 100
```

Toate valorile sunt rotunjite la două zecimale pentru consistență.

Filtrare și Sortare Avansată Utilizatorii pot combina criteriile de căutare cu opțiunile de sortare pentru a găsi rapid filmele dorite. De exemplu, se poate căuta după genul "Drama" și sorta rezultatele după rating descrescător. Funcția de reset permite curățarea rapidă a tuturor filtrelor.

Validări și Mesaje Aplicația validează toate datele de intrare atât pe client cât și pe server. Mesajele de eroare sunt clare și specifice, indicând exact ce trebuie corectat. Mesajele de succes confirmă finalizarea acțiunilor.

---

2. Descrierea Claselor, Metodelor și Atributelor

2.1 Package: com.example.filme.controller

AuthController Acest controller REST gestionează API-urile de autentificare. Clasa conține metode pentru înregistrare și autentificare, folosind `UtilizatorRepository` pentru accesul la baza de date.

Metoda `register` primește un `RegisterRequest` și efectuează următoarele validări: - Verifică dacă username-ul există deja - Verifică dacă email-ul este deja înregistrat - Validează lungimea minimă a parolei

În caz de eroare, aruncă excepții specifice (`UserAlreadyExistsException`, `InvalidRequestException`) care sunt gestionate de `GlobalExceptionHandler`.

Metoda `login` verifică credențialele și returnează datele utilizatorului sau un răspuns 401 Unauthorized.

FilmController Controller REST pentru operațiuni cu filme. Metoda principală `getAllFilms` acceptă parametri opționali pentru căutare și sortare. Implementarea folosește SQL manual cu parametri validați pentru a preveni SQL injection. Criteriile de sortare sunt validate printr-un whitelist pentru a asigura securitatea.

2.2 Package: com.example.filme.controller.page

FilmePageController Acest controller gestionează logica pentru paginile HTML renderizate server-side cu Thymeleaf. Clasa conține metode pentru afișarea listei de filme, detaliilor unui film și gestionarea recenziilor.

Metoda `pageFilme` verifică mai întâi dacă utilizatorul este autentificat prin verificarea sesiunii. Apoi obține lista de filme folosind `FilmJdbcRepository.search()` și adaugă datele în model pentru Thymeleaf.

Metoda `filmPage` obține detaliile complete ale unui film, incluzând genurile, actorii și regizorii prin metoda `findByIdDetailed()`. De asemenea, încarcă toate recenziile pentru film.

Metoda `addRecenzie` validează datele de intrare: - Verifică dacă textul recenziei este completat - Validează rating-ul (1-10) - Verifică dacă utilizatorul are deja o recenzie pentru acest film

După inserarea recenziei, actualizează rating-ul filmului folosind metoda incrementală.

Metoda `editRecenzie` verifică ownership-ul recenziei înainte de a permite editarea. După actualizare, recalculează rating-ul filmului cu noul rating.

Metoda `deleteRecenzie` obține mai întâi rating-ul recenziei care va fi ștearsă, apoi efectuează ștergerea și actualizează rating-ul filmului folosind formula de decrementare.

2.3 Package: com.example.filme.repository

FilmJdbcRepository Repository pentru operațiuni JDBC cu filme. Conține `JdbcTemplate` ca dependență injectată.

Metoda `search` construiește dinamic query-ul SQL bazat pe parametrii primiți. Pentru securitate, criteriile de sortare sunt validate printr-un switch statement care permite doar valorile predefinite. Căutarea după gen, actor sau regizor folosește subquery-uri EXISTS pentru performanță optimă.

Metoda `findByIdDetailed` folosește STRING_AGG pentru a concatena genurile, actorii și regizorii într-un singur query. Această abordare reduce numărul de query-uri la baza de date.

RecenzieJdbcRepository Repository pentru operațiuni cu recenzii.

Metoda `insert` folosește `PreparedStatementCreator` cu `KeyHolder` pentru a obține ID-ul recenziei nou create. Aceasta este necesară pentru a returna ID-ul generat de baza de date.

Metoda `update` primește ID-ul recenziei și noile valori. Validează rating-ul înainte de a executa UPDATE-ul.

Metoda `recalcRatingMediuFilm` implementează formula incrementală de actualizare. SQL-ul folosește ROUND pentru a limita rezultatul la două zecimale:

```
UPDATE Film
SET Rating_mediu = ROUND(Rating_mediu + (CAST(? AS FLOAT) - Rating_mediu) / 100.0, 2)
WHERE ID_Film = ?
```

Metoda `decrementRatingMediuFilm` aplică formula inversă pentru ștergere:

```
UPDATE Film
SET Rating_mediu = ROUND(Rating_mediu - (CAST(? AS FLOAT) - Rating_mediu) / 100.0, 2)
WHERE ID_Film = ?
```

Metoda `findById` este folosită pentru a obține datele unei recenzii înainte de ștergere, necesară pentru actualizarea corectă a rating-ului.

2.4 Package: com.example.filme.dto

FilmDetaliatDto Data Transfer Object pentru filme cu informații complete. Conține atribute pentru toate detaliile filmului: - Informații de bază: ID, titlu, an, durată, descriere - Rating mediu (Double, două zecimale) - Genuri (String concatenat, ex: "Drama, Thriller") - Actori (String concatenat, ex: "Tim Robbins, Morgan Freeman") - Regizori (String concatenat, ex: "Frank Darabont")

RecenzieDto DTO pentru recenzii care include și numele utilizatorului prin JOIN cu tabela Utilizator. Conține ID-ul recenziei, ID-ul filmului, ID-ul utilizatorului, numele utilizatorului, titlul și textul recenziei, rating-ul și data postării.

2.5 Package: com.example.filme.entity

Utilizator Entitate JPA mapată la tabela Utilizator din baza de date. Folosește adnotări JPA pentru mapare: - @Id și @GeneratedValue pentru cheia primară auto-incrementată - @Column cu constrainte pentru unicitate (username, email) - Tipuri de date corespunzătoare (String pentru text, LocalDate pentru date)

2.6 Package: com.example.filme.exception

GlobalExceptionHandler Clasa adnotată cu @RestControllerAdvice care interceptează excepțiile aruncate de controllere. Conține metode pentru diferite

tipuri de excepții:

- `handleInvalidRequest`: Returnează status 400 pentru date invalide
- `handleUserExists`: Returnează status 409 pentru conflicte (username/email duplicat)
- `handleGeneric`: Returnează status 500 pentru erori neașteptate și loghează stack trace-ul

Fiecare metodă returnează un obiect JSON cu timestamp, status code, tip eroare și mesaj detaliat.

---

3. Interfața Grafică cu Utilizatorul

3.1 Pagina de Login

Pagina de login prezintă un formular centrat pe ecran cu câmpuri pentru nume utilizator și parolă. Design-ul folosește un fundal întunecat (#101827) cu un card central care are shadow pentru profunzime vizuală.

Formularul trimite datele prin POST la `/login`. În caz de eroare, se afișează un mesaj sub formular cu text roșu. Link-ul către pagina de înregistrare este plasat sub butonul de submit.

3.2 Pagina de Înregistrare

Similar cu pagina de login, dar cu câmpuri suplimentare pentru email. Validarea se face atât pe client (HTML5 validation) cât și pe server. Mesajele de eroare sunt specifice și indică exact problema (username duplicat, email invalid, etc.).

3.3 Pagina Listă Filme

Pagina principală a aplicației conține o bară de navigare fixată în partea de sus (sticky header) care rămâne vizibilă la scroll. Bara include:

- Titlul aplicației
- Formular de căutare cu mai multe controale:
  - Input text pentru query-ul de căutare
  - Dropdown pentru selectarea câmpului (Titlu/Gen/Actor/Regizor)
  - Dropdown pentru criteriul de sortare
  - Dropdown pentru direcția sortării
  - Buton de căutare
  - Link de reset
- Numele utilizatorului logat
- Buton de logout

Filmele sunt afișate într-un grid responsive care se adaptează la lățimea ecranului (3-4 coloane pe desktop, 1-2 pe tablet, 1 pe mobil). Fiecare card conține titlul filmului, rating-ul, anul și durata. Click pe card navighează la pagina de detalii.

3.4 Pagina Detalii Film

Pagina este împărțită în două secțiuni principale pe desktop (layout cu două coloane):

Secțiunea stângă - Detalii film: - Titlu mare cu rating afișat proeminent - Metadate: an apariție, durată în minute - Descriere completă - Liste pentru genuri, actori și regizori (afișate doar dacă există date) - Lista de recenzii existente

Fiecare recenzie afișează: - Numele autorului și data postării - Rating-ul recenziei - Titlul (dacă există) - Textul complet

Pentru recenziile utilizatorului logat, apar două butoane: - "Editează" (albastru) - deschide formularul de editare inline - "Șterge recenzia" (roșu) - afișează dialog de confirmare

Formular de editare inline: Când utilizatorul dă click pe "Editează", textul recenziei este ascuns și apare un formular pre-populat cu datele existente. Butonul "Editează" se transformă în "Anulează" și își schimbă culoarea în gri. Formularul conține câmpuri pentru titlu, text și rating, plus un buton verde "Salvează".

Secțiunea dreaptă - Adăugare recenzie: Formular pentru adăugarea unei recenzii noi cu câmpuri pentru titlu (opțional), text (obligatoriu) și rating (dropdown 1-10).

Mesajele de eroare și succes apar sub formular cu culori distinctive (roșu pentru erori, verde pentru succes).

3.5 Interacțiuni JavaScript

Aplicația folosește JavaScript vanilla pentru interacțiuni client-side:

Confirmare ștergere: Funcția `confirmDelete` afișează un dialog de confirmare cu mesaj detaliat despre permanența acțiunii. Dacă utilizatorul confirmă, formularul este marcat ca "submitting" pentru a preveni dublul click, iar butonul este dezactivat și textul schimbat în "Se sterge... ".

Toggle editare: Funcția `toggleEdit` comută vizibilitatea formularului de editare. Când formularul este afișat, textul recenziei este ascuns și butonul își schimbă textul și culoarea. Când formularul este ascuns, totul revine la starea inițială.

Anulare editare: Funcția `cancelEdit` ascunde formularul de editare și reafișează textul original al recenziei, resetând și starea butonului.

---

4. Testarea Aplicației

4.1 Metodologie de Testare

Testarea aplicației s-a realizat manual, acoperind scenarii funcționale, de securitate și de performanță. Fiecare test a fost executat de mai multe ori pentru a verifica consistența rezultatelor.

4.2 Teste Funcționale - Autentificare

Test 1: Înregistrare utilizator valid - Pași: Completare formular cu username unic, email valid și parolă - Rezultat: Cont creat cu succes, redirect la pagina de login - Status: Passed

Test 2: Înregistrare username duplicat - Pași: Tentativă de înregistrare cu username existent - Rezultat: Mesaj de eroare "Username deja existent", formularul rămâne afișat - Status: Passed

Test 3: Login cu credențiale corecte - Pași: Introducere username și parolă corecte - Rezultat: Sesiune creată, redirect la lista de filme - Status: Passed

Test 4: Login cu credențiale incorecte - Pași: Introducere parolă greșită - Rezultat: Mesaj de eroare, utilizatorul rămâne pe pagina de login - Status: Passed

Test 5: Protecție rute - Pași: Accesare directă URL /filme fără autentificare - Rezultat: Redirect automat la /login - Status: Passed

4.3 Teste Funcționale - Filme

Test 6: Căutare după titlu - Pași: Introducere text în câmpul de căutare, selectare "Titlu" - Rezultat: Afișare filme care conțin textul în titlu - Status: Passed

Test 7: Căutare după actor - Pași: Introducere nume actor, selectare "Actor" - Rezultat: Afișare filme în care joacă actorul respectiv - Status: Passed

Test 8: Sortare după rating - Pași: Selectare "Rating" și "Desc" - Rezultat: Filme ordonate de la rating mare la mic - Status: Passed

Test 9: Combinare filtre - Pași: Căutare gen "Drama", sortare după an descrescător - Rezultat: Filme de dramă ordonate de la cele mai noi la cele mai vechi - Status: Passed

Test 10: Vizualizare detalii complete - Pași: Click pe un film - Rezultat: Pagină cu toate informațiile: genuri, actori, regizori, descriere - Status: Passed

4.4 Teste Funcționale - Recenzii (CRUD)

Test 11: Adăugare recenzie validă - Pași: Completare formular cu text și rating 8 - Rezultat: Recenzie salvată, mesaj de succes, rating film actualizat - Status: Passed

Test 12: Validare câmp obligatoriu - Pași: Tentativă submit fără text - Rezultat: Mesaj de eroare specific - Status: Passed

Test 13: Prevenire duplicat - Pași: Tentativă adăugare a doua recenzie pentru același film - Rezultat: Mesaj de eroare "Ai deja o recenzie pentru acest film" - Status: Passed

Test 14: Editare recenzie - Pași: Click "Editează", modificare rating de la 8 la 10, salvare - Rezultat: Recenzie actualizată, rating film recalculat, mesaj de succes - Status: Passed

Test 15: Anulare editare - Pași: Click "Editează", apoi "Anulează" - Rezultat: Formular ascuns, date nemodificate - Status: Passed

Test 16: Ștergere cu confirmare - Pași: Click "Șterge", confirmare în dialog - Rezultat: Recenzie ștearsă, rating film ajustat, mesaj de succes - Status: Passed

Test 17: Anulare ștergere - Pași: Click "Șterge", anulare în dialog - Rezultat: Recenzie păstrată - Status: Passed

Test 18: Verificare ownership - Pași: Tentativă de editare a unei recenzii străine prin manipulare URL - Rezultat: Mesaj de eroare, operațiune refuzată - Status: Passed

4.5 Teste Rating Incremental

Test 19: Actualizare la adăugare - Condiții inițiale: Film cu rating 7.50 - Pași: Adăugare recenzie cu rating 10 - Rezultat așteptat: 7.50 + (10-7.50)/100 = 7.52 sau 7.53 - Rezultat obținut: 7.53 (rotunjire la 2 zecimale) - Status: Passed

Test 20: Scădere la ștergere - Condiții inițiale: Film cu rating 7.53 - Pași: Ștergere recenzie cu rating 10 - Rezultat așteptat: Aproximativ 7.50 - Rezultat obținut: 7.50 - Status: Passed

Test 21: Persistență rating - Pași: Ștergere ultimă recenzie a unui film - Rezultat: Rating-ul filmului rămâne neschimbat (nu devine NULL) - Status: Passed

4.6 Teste de Securitate

Test S-01: SQL Injection - Input: `'; DROP TABLE Film; --` în câmpul de căutare - Rezultat: Query parametrizat corect, nicio execuție SQL malițioasă - Status: Passed

Test S-02: XSS Prevention - Input: `<script>alert('XSS')</script>` în text recenzie - Rezultat: Thymeleaf escapează automat HTML, scriptul nu se execută - Status: Passed

Test S-03: Session Management - Pași: Logout, apoi tentativă de accesare pagină protejată - Rezultat: Sesiune invalidată corect, redirect la login - Status: Passed

4.7 Teste de Performanță

Test P-01: Timp încărcare listă - Fără filtre: 50-100ms - Cu filtre complexe (JOIN-uri multiple): 100-200ms - Concluzie: Performanță acceptabilă pentru volume moderate de date

Test P-02: Eficiență rating incremental - Metoda incrementală: O(1) - un singur UPDATE - Metoda tradițională (AVG): O(n) - parcurgere toate recenziile - Concluzie: Metoda incrementală oferă performanță superioară

4.8 Rezumat Teste

Categorie	Total Teste	Passed	Failed
Autentificare	5	5	0
Filme	5	5	0
Recenzii CRUD	8	8	0
Rating Incremental	3	3	0
Securitate	3	3	0
Performanță	2	2	0
Total	**26**	**26**	**0**

---

5. Îmbunătățiri Propuse

5.1 Implementare Paginare

Sistemul actual încarcă toate filmele dintr-o singură interogare. Pentru baze de date cu mii de filme, acest lucru poate cauza probleme de performanță. Implementarea paginării ar permite afișarea unui număr limitat de rezultate per pagină (de exemplu, 20-30 filme).

Implementarea tehnică ar folosi clauza SQL Server OFFSET...FETCH:

```
SELECT ... FROM Film
WHERE ...
ORDER BY ...
OFFSET @PageSize * (@PageNumber - 1) ROWS
FETCH NEXT @PageSize ROWS ONLY
```

Interfața ar include controale de navigare (Previous, 1, 2, 3, ..., Next) și un indicator al paginii curente. Această îmbunătățire ar reduce timpul de încărcare cu 70-80% pentru liste mari și ar îmbunătăți experiența utilizatorului.

5.2 Dashboard cu Statistici și Rapoarte

O pagină dedicată statisticilor ar oferi insight-uri valoroase despre conținutul aplicației. Funcționalitățile propuse includ:

Top filme după rating: Afișarea celor mai bine cotate 10 filme folosind sortare și limitare. Implementarea ar demonstra utilizarea Stream API și Comparators.

Utilizatori activi: Identificarea utilizatorilor cu cele mai multe recenzii prin GROUP BY și COUNT în SQL, sau prin Stream.collect(Collectors.groupingBy()) în Java.

Distribuție rating-uri: Histogram care arată câte filme au rating în diferite intervale (1-2, 2-3, etc.). Aceasta ar demonstra procesare de date și agregare.

Genuri populare: Analiza frecvenței genurilor prin JOIN-uri și agregare. Rezultatele ar putea fi afișate într-un grafic sau tabel.

Filme fără recenzii: Identificarea filmelor care nu au primit nicio recenzie, util pentru moderatori.

Această funcționalitate ar demonstra competențe în procesare de date (max, min, count, groupBy) și ar adăuga valoare practică aplicației.

5.3 Sistem de Favorite și Wishlist

Utilizatorii ar putea marca filme ca "favorite" sau "de văzut", creând liste personalizate. Implementarea ar necesita o tabelă suplimentară:

```
CREATE TABLE User_Favorite (
    ID INT PRIMARY KEY IDENTITY,
    ID_Utilizator INT FOREIGN KEY REFERENCES Utilizator(ID_Utilizator),
    ID_Film INT FOREIGN KEY REFERENCES Film(ID_Film),
    Type VARCHAR(20) CHECK (Type IN ('favorite', 'watchlist')),
    Data_adaugarii DATE DEFAULT GETDATE(),
    CONSTRAINT UQ_User_Film_Type UNIQUE(ID_Utilizator, ID_Film, Type)
)
```

Interfața ar include butoane toggle pe cardurile de filme și o pagină dedicată "Filmele Mele" cu tab-uri pentru diferite categorii. Funcționalitatea de export în CSV sau JSON ar permite utilizatorilor să salveze listele lor.

5.4 Algoritm de Recomandări

Un sistem simplu de recomandări ar analiza preferințele utilizatorului bazat pe rating-urile acordate și ar sugera filme similare. Algoritmul ar funcționa astfel:

1. Identificare genuri preferate: extragere genuri din filmele cu rating > 7 acordat de utilizator
2. Calculare frecvență genuri pentru a determina top 3 genuri favorite
3. Căutare filme cu genuri similare pe care utilizatorul nu le-a văzut
4. Sortare după rating mediu descrescător
5. Returnare top 10 recomandări

Implementarea ar demonstra utilizarea avansată a Stream API:

```java
List<String> preferredGenres = userReviews.stream()
    .filter(r -> r.getRating() > 7)
    .map(r -> r.getFilm().getGenuri())
    .flatMap(g -> Arrays.stream(g.split(", ")))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(3)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

5.5 Export și Raportare

Funcționalitatea de export ar permite utilizatorilor să salveze date în formate standard:

Export PDF: Generare document PDF cu recenziile utilizatorului folosind librării precum iText sau Apache PDFBox. Documentul ar include header cu logo, informații utilizator și listă formatată de recenzii.

Export Excel: Creare fișier Excel cu mai multe sheet-uri (Filme, Recenzii, Statistici) folosind Apache POI. Acest format este util pentru analiză ulterioară în alte aplicații.

Export JSON: Serializare date în format JSON pentru interoperabilitate cu alte sisteme sau pentru backup.

Implementarea ar include endpoint-uri REST care returnează fișierele cu header-ele HTTP corespunzătoare pentru download:

```java
@GetMapping("/export/reviews/pdf")
public ResponseEntity<byte[]> exportReviewsPdf(HttpSession session) {
    int userId = (Integer) session.getAttribute("userId");
    List<RecenzieDto> reviews = recenzieRepo.findByUserId(userId);
    byte[] pdfBytes = pdfGenerator.generateReviewPdf(reviews);

    return ResponseEntity.ok()
        .header("Content-Disposition", "attachment; filename=recenzii.pdf")
        .contentType(MediaType.APPLICATION_PDF)
        .body(pdfBytes);
}
```

Anexe

Tehnologii Utilizate

Backend Framework: Spring Boot 3.x cu Java 17 oferă un framework modern pentru dezvoltare web enterprise. Spring MVC gestionează rutarea și procesarea

request-urilor HTTP, iar Spring JDBC simplifică accesul la baza de date.

Template Engine: Thymeleaf permite renderare server-side a paginilor HTML cu sintaxă naturală și integrare strânsă cu Spring.

Baza de Date: SQL Server oferă funcționalități avansate precum STRING_AGG pentru agregare și suport excelent pentru tranzacții.

Frontend: HTML5, CSS3 și JavaScript vanilla asigură compatibilitate largă și performanță optimă fără dependențe externe.

Build Tool: Maven gestionează dependențele și procesul de build.

Structura Bazei de Date

Baza de date folosește un model relațional normalizat:

Tabele principale: - Film: Stochează informații despre filme (titlu, an, durată, descriere, rating) - Utilizator: Conturi utilizatori cu credențiale și date personale - Recenzie: Recenzii ale utilizatorilor pentru filme

Tabele auxiliare: - Gen, Actor, Regizor: Entități separate pentru normalizare

Tabele de legătură (Many-to-Many): - Film_Gen: Asociere filme-genuri - Film_Actor: Asociere filme-actori - Film_Regizor: Asociere filme-regizori

Această structură permite flexibilitate maximă și evită redundanța datelor.

Arhitectura Aplicației

Aplicația folosește arhitectura MVC (Model-View-Controller) cu separare clară a responsabilităților:

Presentation Layer: Controllere Spring MVC care procesează request-urile HTTP și returnează view-uri Thymeleaf sau răspunsuri JSON.

Business Layer: Logica de business este implementată în controllere și servicii, incluzând validări și procesare de date.

Data Access Layer: Repository-uri care encapsulează accesul la baza de date folosind Spring JDBC Template.

Database Layer: SQL Server stochează persistent datele aplicației.

Fluxul de date este unidirecțional: Browser → Controller → Repository → Database și înapoi.

Concluzie

Aplicația de gestionare filme și recenzii îndeplinește cerințele proiectului prin implementarea completă a operațiunilor CRUD pentru recenzii, un sistem avansat

de rating incremental și funcționalități de căutare și sortare.

Aspectele tehnice notabile includ: - Securitate prin parametrizare SQL și validare input - Performanță optimizată prin actualizare incrementală a rating-ului - Interfață intuitivă cu feedback clar pentru utilizator - Cod bine structurat cu separare clară a responsabilităților

Proiectul demonstrează competențe în programare orientată pe obiecte, dezvoltare web cu Spring Boot, lucru cu baze de date relaționale și design de interfețe utilizator.

Autor: Sipanu Eduard Nicusor **Data:** 12 Ianuarie 2026